

# Informe de Análisis de Seguridad

Análisis estático de código fuente

**Proyecto:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples

**Fecha:** 02/08/2026 12:56

**Archivos analizados:** 2

**Total de hallazgos:** 13

**Duración del escaneo:** 0.0 segundos

| Severidad | Cantidad |
|-----------|----------|
| ■ Crítica | 6        |
| ■ Alta    | 5        |
| ■ Media   | 2        |
| ■ Baja    | 0        |

# Hallazgos de Seguridad

## 1. [CRÍTICA] Command Injection — <font color='Color(.862745,.078431,.235294,1)'+CWE-78</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\python\vulnerable.py:26

**Confianza:** Media | **Lenguaje:** python

**Descripción:** Posible command injection: ejecución de comandos del sistema con datos de usuario sin sanitizar:  
``subprocess.call("ping -c 1 " +``

**Código:**

```
host = request.GET.get("host") result = subprocess.call("ping -c 1 " + host, shell=True) return result
```

**Remediación:**

Evita ejecutar comandos del sistema con input de usuario. Si es inevitable, usa listas de argumentos en vez de strings (ej: ``subprocess.run(['cmd', arg])`` en vez de ``os.system(f'cmd {arg}')``). JS: evita ``child_process.exec()`` con input de usuario. Usa ``execFile()`` con argumentos como array. PHP: evita ``exec()``, ``system()``, ``passthru()``, ``shell_exec()`` con input de usuario. Usa ``escapeshellarg()``. Java: evita ``Runtime.exec()`` con strings concatenados.

## 2. [CRÍTICA] Hardcoded Secrets — <font color='Color(.862745,.078431,.235294,1)'+CWE-798</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:5

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Secreto hardcodeado detectado en el código fuente: ``API_SECRET = "***REDACTED***"``

**Código:**

```
const API_SECRET = "super-secret-token-1234567890"; const DB_CONNECTION = "mongodb://admin:password123@localhost:27017/mydb";
```

**Remediación:**

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

## 3. [CRÍTICA] Hardcoded Secrets — <font color='Color(.862745,.078431,.235294,1)'+CWE-798</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\python\vulnerable.py:6

**Confianza:** Media | **Lenguaje:** python

**Descripción:** Secreto hardcodeado detectado en el código fuente: ``API_KEY = "***REDACTED***"``

**Código:**

```
API_KEY = "sk-1234567890abcdef1234567890abcdef" DB_PASSWORD = "admin123"
```

**Remediación:**

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

## 4. [CRÍTICA] Hardcoded Secrets — <font color='Color(.862745,.078431,.235294,1)'+CWE-798</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\python\vulnerable.py:7

**Confianza:** Media | **Lenguaje:** python

**Descripción:** Secreto hardcodeado detectado en el código fuente: ``PASSWORD = "***REDACTED***"``

**Código:**

```
API_KEY = "sk-1234567890abcdef1234567890abcdef" DB_PASSWORD = "admin123"
```

#### Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

### 5. [CRÍTICA] Sql Injection — <font color='Color(.862745,.078431,.235294,1)'+CWE-89</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:10

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `SELECT \* FROM users WHERE id = "" + userId + ""`;

#### Código:

```
const userId = req.query.id; const query = "SELECT * FROM users WHERE id = '" + userId + "'";
db.query(query);
```

#### Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query('SELECT \* FROM users WHERE id = ?', [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

### 6. [CRÍTICA] Sql Injection — <font color='Color(.862745,.078431,.235294,1)'+CWE-89</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\python\vulnerable.py:13

**Confianza:** Media | **Lenguaje:** python

**Descripción:** Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `SELECT \* FROM users WHERE user="" + user + "" AND pass="" + passwd + ""`

#### Código:

```
passwd = request.GET.get("pass") query = "SELECT * FROM users WHERE user='" + user + "' AND
pass='" + passwd + "'" cursor.execute(query)
```

#### Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query('SELECT \* FROM users WHERE id = ?', [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

### 7. [ALTA] Dynamic Exec — <font color='Color(1,.270588,0,1)'+CWE-95</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:24

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Ejecución dinámica de código detectada (eval/exec/Function): `eval(`

#### Código:

```
const code = req.query.code; eval("var user = " + code + ";"); res.json(user);
```

#### Remediación:

Evita `eval()`, `exec()`, `Function()` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita `eval()` y `exec()`. Si necesitas evaluar expresiones simples, usa `ast.literal\_eval()`. JS/TS: no uses `eval()` ni `new Function()`. Para parsear JSON usa `JSON.parse()`. PHP: evita `eval()`, `preg\_replace()` con modificador /e, `create\_function()`. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

### 8. [ALTA] Dynamic Exec — <font color='Color(1,.270588,0,1)'+CWE-95</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:30

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Ejecución dinámica de código detectada (eval/exec/Function): ``exec(``

**Código:**

```
const host = req.query.host; const child = require("child_process").exec("ping -c 1 " + host); });
```

**Remediación:**

Evita ``eval()``, ``exec()``, ``Function()`` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita ``eval()`` y ``exec()``. Si necesitas evaluar expresiones simples, usa ``ast.literal_eval()``. JS/TS: no uses ``eval()`` ni ``new Function()``. Para parsear JSON usa ``JSON.parse()``. PHP: evita ``eval()``, ``preg_replace()`` con modificador `/e`, ``create_function()``. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

## 9. [ALTA] Insecure Deserialization — **CWE-502**

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\python\vulnerable.py:32

**Confianza:** Media | **Lenguaje:** python

**Descripción:** Deserialización de datos no confiables detectada: ``pickle.loads(``

**Código:**

```
try: return pickle.loads(data) except Exception:
```

**Remediación:**

No deserialices datos que provengan de fuentes no confiables (usuarios, red). Python: evita ``pickle.loads()`` con datos externos. Usa JSON para serialización segura. JS: ten cuidado con ``JSON.parse()`` cuando los datos no están validados; no uses ``eval()``. PHP: evita ``unserialize()`` con datos de usuario. Usa JSON. Java: usa whitelists de clases con ``ObjectInputStream``, o evita la deserialización de Java nativo. Prefiere formatos de datos seguros como JSON, Protocol Buffers, o MessagePack.

## 10. [ALTA] Xss — **CWE-79**

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:19

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Posible XSS (Cross-Site Scripting) detectada: renderizado de contenido no sanitizado en el DOM: ``innerHTML =``

**Código:**

```
const html = '<div>welcome <b>' + user + '</b></div>';  
document.getElementById("content").innerHTML = html; });
```

**Remediación:**

Escapa siempre el contenido antes de insertarlo en el DOM. JS: no uses ``innerHTML`` / ``dangerouslySetInnerHTML`` con datos de usuario. Usa ``textContent`` o sanitización con DOMPurify. React: evita ``dangerouslySetInnerHTML`` a menos que sanitices con DOMPurify. Python templates: usa autoescape (Django, Jinja2). PHP: usa ``htmlspecialchars()`` al renderizar en HTML. Valida y sanitiza cualquier input de usuario antes de mostrarlo.

## 11. [ALTA] Xss — **CWE-79**

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:24

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Posible XSS (Cross-Site Scripting) detectada: renderizado de contenido no sanitizado en el DOM: ``eval("var user = " + code + "``

**Código:**

```
const code = req.query.code; eval("var user = " + code + ";"); res.json(user);
```

**Remediación:**

Escapa siempre el contenido antes de insertarlo en el DOM. JS: no uses ``innerHTML`` / ``dangerouslySetInnerHTML`` con datos de usuario. Usa ``textContent`` o sanitización con DOMPurify. React: evita ``dangerouslySetInnerHTML`` a menos que sanitices con DOMPurify. Python templates: usa autoescape (Django, Jinja2). PHP: usa ``htmlspecialchars()`` al renderizar en HTML. Valida y sanitiza cualquier input de usuario antes de mostrarlo.

## 12. [MEDIA] Insecure Crypto — <font color='Color(1,647059,0,1)'+CWE-327</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\js\vulnerable.js:34

**Confianza:** Media | **Lenguaje:** javascript

**Descripción:** Uso de algoritmo criptográfico débil u obsoleto detectado: `crypto.createHash("md5")`

**Código:**

```
function hashPassword(password) { return crypto.createHash("md5").update(password).digest("hex");
}
```

**Remediación:**

No uses MD5 o SHA1 para hashing de contraseñas o firmas. Para contraseñas: usa bcrypt, scrypt, argon2, o PBKDF2 con salt. Para integridad: usa SHA-256 o SHA-3. Python: usa `hashlib.sha256()` en vez de `hashlib.md5()`, `bcrypt` para contraseñas. JS: usa `crypto.createHash('sha256')` en vez de `md5`. No implementes criptografía propia, usa librerías estándar y auditadas.

## 13. [MEDIA] Insecure Crypto — <font color='Color(1,647059,0,1)'+CWE-327</font>

**Archivo:** C:\Users\oscar\Desktop\Proyectos\análisis\_código\tests\samples\python\vulnerable.py:38

**Confianza:** Media | **Lenguaje:** python

**Descripción:** Uso de algoritmo criptográfico débil u obsoleto detectado: `hashlib.md5`

**Código:**

```
def hash_password(passwd): return hashlib.md5(passwd.encode()).hexdigest()
```

**Remediación:**

No uses MD5 o SHA1 para hashing de contraseñas o firmas. Para contraseñas: usa bcrypt, scrypt, argon2, o PBKDF2 con salt. Para integridad: usa SHA-256 o SHA-3. Python: usa `hashlib.sha256()` en vez de `hashlib.md5()`, `bcrypt` para contraseñas. JS: usa `crypto.createHash('sha256')` en vez de `md5`. No implementes criptografía propia, usa librerías estándar y auditadas.

---